

Lab 04 - Expected evidence

Timestamps, counter values, IPs and node names will differ. The marker strings and the structure are the evidence.

1. Namespace, ConfigMaps, backend

```
$ kubectl -n kcna-lab04 create configmap access-seed --from-file=data/access-seed.log
configmap/access-seed created

$ kubectl -n kcna-lab04 create configmap tariff-quote --from-file=data/tariff-quote.json
configmap/tariff-quote created

$ kubectl apply -f manifests/10-tariff-backend.yaml
pod/tariff-backend created
service/tariff-backend created

$ kubectl -n kcna-lab04 get endpointslices -l kubernetes.io/service-name=tariff-backend
```

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
tariff-backend-9k4xd	IPv4	80	10.244.0.19	22s

EndpointSlice - not the deprecated v1 Endpoints - is the current way to see a Service's backends.

2. Sidecar

```
$ kubectl -n kcna-lab04 get pod depot-sidecar
```

NAME	READY	STATUS	RESTARTS	AGE
depot-sidecar	2/2	Running	0	25s

READY 2/2 - two containers, both ready.

```
$ kubectl -n kcna-lab04 logs depot-sidecar
error: a container name must be specified for pod depot-sidecar, choose one of: [app shipper]
```

```
$ kubectl -n kcna-lab04 logs depot-sidecar -c app --tail=3
[app] seeded 16 historical rows
```

```
$ kubectl -n kcna-lab04 logs depot-sidecar -c shipper --tail=6
2026-09-05T08:00:37Z depot=MF-SIN-02 path=/api/trailers status=404 bytes=64 latency_ms=11
2026-09-05T08:00:40Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=37
2026-09-05T08:00:43Z depot=MF-SIN-02 path=/api/tariff status=503 bytes=0 latency_ms=3002
2026-09-05T08:00:46Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=42
2026-09-05T16:12:07Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=44
2026-09-05T16:12:10Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=44
```

The first four lines came from data/access-seed.log; the last two were generated live. The shipper wrote none of them - it only read the shared volume.

```
$ kubectl -n kcna-lab04 exec depot-sidecar -c shipper -- ls -l /var/log/depot
total 4
-rw-r--r-- 1 1000 3000 2814 Sep 5 16:12 access.log
```

```
$ kubectl -n kcna-lab04 exec depot-sidecar -c shipper -- sh -c 'echo x >> /var/log/depot/
access.log'
sh: can't create /var/log/depot/access.log: Read-only file system
command terminated with exit code 1
```

The shipper's mount is readOnly: true; the app's is not. Same volume, different permissions per container.

3. Adapter

```
$ kubectl -n kcna-lab04 logs depot-adapter -c app --tail=2
2026-09-05T16:14:22Z depot=MF-SIN-02 path=/api/bookings status=404 bytes=812 latency_ms=44
2026-09-05T16:14:24Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=44
```

```
$ kubectl -n kcna-lab04 logs depot-adapter -c adapter --tail=12
# HELP depot_http_requests_total Depot Portal requests by status code.
# TYPE depot_http_requests_total counter
depot_http_requests_total{depot="MF-SIN-02",code="200"} 24
depot_http_requests_total{depot="MF-SIN-02",code="201"} 4
depot_http_requests_total{depot="MF-SIN-02",code="404"} 4
depot_http_requests_total{depot="MF-SIN-02",code="503"} 4
# HELP depot_http_requests_observed_total All parsed log rows.
# TYPE depot_http_requests_observed_total counter
depot_http_requests_observed_total{depot="MF-SIN-02"} 36
```

Same input, different *shape*. The application still speaks key=value; the platform now receives Prometheus text exposition format.

4. Ambassador

```
$ kubectl -n kcna-lab04 get pod depot-ambassador
NAME          READY   STATUS    RESTARTS   AGE
depot-ambassador 2/2     Running   0           41s
```

```
$ kubectl -n kcna-lab04 logs depot-ambassador -c app --tail=4
[app] I only know about http://127.0.0.1:9000
[app] quote via ambassador: "bay_hour":51.00,
[app] quote via ambassador: "bay_hour":51.00,
[app] quote via ambassador: "bay_hour":51.00,
```

```
$ kubectl -n kcna-lab04 logs depot-ambassador -c ambassador --tail=3
127.0.0.1 - - [05/Sep/2026:16:16:02 +0000] "GET /tariff-quote.json HTTP/1.1" 200 341 "-" "wget"
127.0.0.1 - - [05/Sep/2026:16:16:12 +0000] "GET /tariff-quote.json HTTP/1.1" 200 341 "-" "wget"
127.0.0.1 - - [05/Sep/2026:16:16:22 +0000] "GET /tariff-quote.json HTTP/1.1" 200 341 "-" "wget"
```

The client address is 127.0.0.1 - the app container and the ambassador share a loopback interface because they share a network namespace.

```
$ kubectl -n kcna-lab04 exec depot-ambassador -c app -- wget -qO- http://127.0.0.1:9000/
ambassador-health
ambassador ok
```

5. Native sidecar

```
$ kubectl -n kcna-lab04 get pod depot-native-sidecar
NAME                READY   STATUS    RESTARTS   AGE
depot-native-sidecar 1/1     Running   0           33s
```

READY 1/1 counts only spec.containers. The native sidecar lives in initContainers and is not part of that ratio - but it is running:

```
$ kubectl -n kcna-lab04 get pod depot-native-sidecar \
  -o jsonpath='{range .status.initContainerStatuses[*]}{.name}{"\t"}{.state}{"\n"}{end}'
prepare█{"terminated":{"containerID":"containerd://...", "exitCode":0, "finishedAt":"2026-09-05T16:17:41Z", "reason":"Completed", "startedAt":"2026-09-05T16:17:41Z"}}
shipper█{"running":{"startedAt":"2026-09-05T16:17:43Z"}}
```

prepare is terminated / Completed; shipper is running. That is the whole difference restartPolicy: Always makes on an init container.

```
$ kubectl -n kcna-lab04 logs depot-native-sidecar -c shipper --tail=3
[shipper] native sidecar up BEFORE the app container
2026-09-05T08:00:46Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=42
2026-09-05T16:17:48Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=44
```

```
$ kubectl -n kcna-lab04 describe pod depot-native-sidecar | sed -n '/^Init Containers:\/,\/
^Containers:\/p' | head -n 12
Init Containers:
  prepare:
    Container ID:   containerd://6f1c...
    Image:          busybox:1.36
    State:          Terminated
      Reason:       Completed
      Exit Code:     0
  shipper:
    Container ID:   containerd://a3b8...
    Image:          busybox:1.36
    Restart Policy: Always
    State:          Running
```

6. Failure injection - independent volumeMounts

```
$ kubectl apply -f manifests/90-sidecar-broken-volume.yaml
The Pod "depot-sidecar-broken" is invalid: spec.containers[1].volumeMounts[0].name: Not found:
"aplogs"
```

Nothing was created:

```
$ kubectl -n kcna-lab04 get pod depot-sidecar-broken
Error from server (NotFound): pods "depot-sidecar-broken" not found
```

7. Verification script

```
$ bash verification/checks.sh

== 0. Cluster reachability
[PASS] kubectl can reach the API server

== 1. Namespace and ConfigMaps built from data/
[PASS] namespace kcna-lab04 exists
[PASS] ConfigMap access-seed exists (from data/access-seed.log)
[PASS] ConfigMap tariff-quote exists (from data/tariff-quote.json)

== 2. Backend service for the ambassador
[PASS] Pod tariff-backend is Ready
[PASS] Service tariff-backend has an EndpointSlice backend (10.244.0.19)

== 3. SIDECAR - depot-sidecar
[PASS] depot-sidecar has 2 containers (app, shipper)
[PASS] depot-sidecar is Ready
[PASS] app seeded 16 rows from data/access-seed.log into the shared emptyDir
[PASS] shipper re-emitted the seeded rows it never wrote - the volume is shared

== 4. ADAPTER - depot-adapter
[PASS] depot-adapter is Ready
[PASS] adapter emits Prometheus TYPE metadata
[PASS] adapter derived a 503 counter from the app's key=value log

== 5. AMBASSADOR - depot-ambassador
[PASS] depot-ambassador is Ready
[PASS] app reached the remote service through 127.0.0.1:9000
[PASS] ambassador access log shows the proxied 200

== 6. NATIVE SIDECAR - depot-native-sidecar
[PASS] initContainer 'shipper' declares restartPolicy: Always (native sidecar)
[PASS] depot-native-sidecar is Ready
[PASS] the native sidecar is still RUNNING while the app runs (started 2026-09-05T16:17:43Z)
[PASS] native sidecar logged its start banner

== 7. Failure injection: independent volumeMounts
[PASS] the API server rejects the broken manifest with the expected field-path error

Result: 20 passed, 0 failed
```

Section 4 needs the adapter's 15-second loop to have fired at least once. If it

fails, wait 20 seconds and re-run.